ProfAI — Talking-Avatar (HeyGen Lip-Sync)

How it works, why it's slow, and how to fix it · Prepared for IT department review · 2026-08-07

1. One-paragraph summary

The talking avatar is produced by an **external cloud service, HeyGen**. Our app sends HeyGen the scene's narration audio plus the chosen avatar; HeyGen renders a lip-synced presenter video **on their servers** and we download it and overlay it onto the slide. The slow part is **HeyGen's own render queue (~1–5 minutes per clip, regardless of how short the clip is)** — it is not our code, our CPU, or the network. Everything our app can control has already been optimized around that fixed cost.

2. The files involved (where lip-sync lives)

Backend (Azure Functions, TypeScript — source in `api/src`):

File	Role
`api/src/lib/heygenAvatar.ts`	**The HeyGen client.** Uploads audio (POST /v3/assets), submits the render job (POST /v3/videos), polls status, downloads the finished MP4, and caches it. Read this file first.
`api/src/functions/agents/generateHeyGenAvatar.ts`	HTTP endpoint the app calls to start a scene's video. Renders the slide clip locally, then hands off to HeyGen asynchronously (returns a video_id immediately).
`api/src/functions/agents/pollHeyGenVideo.ts`	Called repeatedly by the app to check a job. When HeyGen reports completed, downloads the clip, overlays the avatar onto the slide, saves the result.
`api/src/functions/agents/heygenWebhook.ts`	Alternative to polling (currently unused in practice). HeyGen can call this URL when a render finishes. Requires a public URL.
`api/src/functions/agents/mergeModuleVideo.ts`	After every scene has a rendered video, concatenates them into one module video, with a timed pause between scenes.
`api/src/lib/ffmpegVideo.ts`	Local video work with ffmpeg: renders the slide+audio clip, overlays the avatar box, concatenates scenes.
`api/src/lib/slideRenderer.ts`	Turns a slide design into the still image used in the clip.
`api/prisma/schema.prisma`	Stores avatarVideoUrl (per scene) and fullVideoUrl (per module).
`api/.env`	Holds HEYGEN_API_KEY.

Frontend (React — `src`):

File	Role
`src/components/workspace/FinalVideoPanel.jsx`	The "Final Video" step. Kicks off generation, throttles local renders, polls each HeyGen job, then triggers the merge.
`src/components/workspace/VideoEditingPanel.jsx`	The "Module Editing" step. Scene transitions/approval, plus a "Regenerate" button to test-render a module's talking avatar without leaving to Final Video.
`src/services/agents.js`	The API calls: runHeyGenAvatar, pollHeyGen, runMergeModuleVideo.
`src/components/workspace/CastingSettings.jsx`	Where the avatar + voice are chosen (required before any render).

3. The flow, step by step (what happens when you click "Generate")

For **one scene**:

1. **Render the slide clip locally** (generateHeyGenAvatar.ts → ffmpegVideo.ts). Slide image + narration audio → a short MP4. ~6–25 seconds on our machine (ffmpeg).
2. **Upload the audio to HeyGen** (heygenAvatar.ts › uploadAudioAsset). ~1–2 s.
3. **Submit the render job** (heygenAvatar.ts › createAvatarVideo, POST /v3/videos). Returns a video_id immediately — the app does **not** wait here.
4. **HeyGen renders the avatar on their servers. ~1–5 minutes.** This is the dominant cost and is fixed by HeyGen regardless of clip length.
5. **The app polls every 5 seconds** (pollHeyGenVideo.ts) asking "is it done?" until HeyGen says completed (or failed).
6. **Download + overlay** the finished avatar onto the slide clip (ffmpeg). ~2–4 s.
7. Once **every** scene in the module is done, **merge** them into the module video (mergeModuleVideo.ts), with a 5-second pause inserted between scenes. A few seconds.

There is a **cache** (heygenAvatar.ts): a finished avatar clip is stored on disk keyed by a hash of (narration audio + avatar + style). Re-generating a scene whose narration and avatar didn't change reuses the cached clip **instantly** and skips HeyGen entirely.

4. Where the time actually goes (measured from our logs)

Step	Time	On whose machine
Local slide render (ffmpeg)	~6–25 s / scene	Ours
Audio upload + job submit	~2–5 s	Ours → HeyGen
HeyGen avatar render	**~1–5 min / scene**	**HeyGen (external)**
Poll interval	every 5 s (cheap)	Ours

Step	Time	On whose machine
Download + overlay	~2–4 s	Ours
Merge module	a few s	Ours

So for a module of N scenes, wall-clock is roughly: **(local renders) + (~2–5 min for the HeyGen queue, overlapped across scenes) + (merge)**. The HeyGen minutes are the floor.

Things that made it feel even slower (now fixed or identified):

- **CPU thrash from too much parallelism (fixed).** Too many local ffmpeg renders at once fought for CPU, ballooning each submit from ~25 s to 112–164 s. Local renders are now throttled (2 at a time) while HeyGen polling stays fully parallel.
- **Polling a failed job forever (fixed).** A failed HeyGen render used to keep polling for ~15 minutes because it only stopped on completed. It now stops immediately on failed.
- **All-or-nothing merge (open item).** If one scene's HeyGen render fails, the whole module currently produces nothing — see Solution D.
- **AzureWebJobsStorage "Unhealthy" log noise.** The Functions host repeatedly logs that it can't reach its storage backend. Cosmetic in local dev (this app only uses HTTP-triggered functions, no queues/timers that need that storage) — but worth a valid AzureWebJobsStorage connection string before production.

4b. Recent incidents — HeyGen's v3 API migration (2026-08-07)

HeyGen is retiring the v1/v2 endpoints we originally integrated against (upload.heygen.com/v1/asset, v2/video/generate, v1/video_status.get) on **2026-11-01**. We migrated to v3 (POST /v3/assets, POST /v3/videos, GET /v3/videos/{id}) ahead of that deadline, and hit two undocumented-in-practice quirks worth raising with HeyGen's team directly:

1. **v3 silently defaults to HeyGen's newest "Avatar IV" render engine** when the engine field is omitted — but Avatar IV only works with avatars specifically built as Digital Twin/Photo Avatar "looks", not the broad ~1,200-avatar catalog most accounts (including ours) actually pick from. Every render against a regular avatar returned a 400 ("This video avatar does not support Avatar IV video generation"), and the app silently fell back to a voice-only clip instead of surfacing a clear error. **Fix:** we now explicitly request the older, broadly-compatible avatar_iii engine, with one retry on avatar_iv if a specific avatar rejects III.
2. **The engine field must be an object, not a string** — engine: "avatar_iii" returns a 400 ("Input should be a valid dictionary or object to extract fields from"); the correct shape is engine: { "type": "avatar_iii" }. Not obvious from the field name alone.

Both are fixed in heygenAvatar.ts and confirmed working end-to-end. Flagging these because they'd likely trip up any other engineer integrating HeyGen's v3 API the same way we first did.

5. Why it can't be "under 1 minute" with real lip-sync

The ~1–5 min per clip happens **inside HeyGen**, on their servers. No change to our code, CPU, or network removes it — we can only stop stacking it, overlap it, and cache it (all done). The only ways to go faster are to change **what** we ask HeyGen to do, or to not use HeyGen for every clip.

6. Solutions (ranked, to discuss with IT)

A. Use HeyGen webhooks instead of polling *(biggest architectural win)*

heygenWebhook.ts already exists. Instead of asking "is it done?" every 5 s, HeyGen calls our URL the instant a render finishes. Removes polling load, makes completion instant to detect. **Needs a publicly reachable URL** (a deployed API, or a tunnel like ngrok in dev) and the webhook registered in the HeyGen dashboard. On localhost, HeyGen can't reach us, which is why we poll today.

B. Render scenes truly in parallel across the whole course

HeyGen renders multiple jobs concurrently on their side. We already submit per scene; confirming our HeyGen plan's concurrency limit means N scenes finish in ~one render window instead of N windows. Depends on the HeyGen plan tier — a question for the meeting.

C. Shorten narration per scene

HeyGen bills and renders per clip; fewer/shorter clips = fewer render windows. Splitting long scripts differently is a content lever, not a code one.

D. Make the merge resilient (partial video)

Change mergeModuleVideo.ts to skip a scene that failed to render and stitch the rest, so one bad scene doesn't waste the whole module. Small code change.

E. Offer a "static avatar / voice-only" fast path for previews

Already added a Fast-preview toggle (voice-only, no HeyGen, seconds per clip). Optionally add a static-avatar-image mode (avatar shown, no lip movement) for instant drafts, keeping HeyGen only for the final export.

F. Keep/improve the cache

Already caches finished clips by narration+avatar hash. Re-running an unchanged scene is instant. Worth confirming the cache survives restarts and cleanups.

7. Key questions for the IT / HeyGen discussion

- What is our **HeyGen plan tier**, and how many videos can render **concurrently**?
- What is HeyGen's **typical and worst-case render time** on our plan?
- Can we register a **webhook** (needs a public HTTPS endpoint for the API)?
- Are the recent **failed renders** a plan/quota limit, an avatar/voice rejection, or clip length?
- Where will this run in production (the machine doing the local ffmpeg renders matters for step 1)?

8. Bottom line

Nothing here is broken by design — the architecture is already async, cached, throttled, and overlaps local work with HeyGen. The irreducible cost is **HeyGen's 1–5 min server render per clip**. The highest-value change is **switching from polling to webhooks** (code already scaffolded) once the API has a public URL, plus confirming our HeyGen concurrency tier.